PROJECT REPORT

RAG-Based Medical Chatbot using LLMs, LangChain, Pinecone, and AWS

1. Introduction

With the rapid growth of medical literature and digital health records, accessing accurate and context-aware medical information has become increasingly challenging. Traditional search systems often fail to provide precise, conversational, and evidence-grounded answers.

This project addresses the problem by building a **Retrieval-Augmented Generation (RAG)–based Medical Chatbot** that combines:

- **Vector search** for factual grounding
- **Large Language Models (LLMs)** for natural language understanding
- **Conversation memory** for multi-turn dialogue
- **Cloud-native deployment** for scalability and reliability

The system enables users to ask medical questions and receive **contextually accurate responses grounded in trusted medical documents**, rather than relying purely on a language model's pre-trained knowledge.

---

2. Project Objectives

- Build an **end-to-end RAG pipeline** for medical document question answering
- Enable **semantic search** over medical PDFs using embeddings
- Integrate **Gemini LLM** for high-quality natural language responses
- Maintain **conversation context** across multiple user queries
- Deploy the application using **Docker, AWS EC2, and GitHub Actions CI/CD**
- Ensure **secure, scalable, and production-ready architecture**

---

3. System Architecture Overview

The system follows a **two-phase RAG architecture**:

Phase 1: Knowledge Base Creation (Offline)

1. PDF ingestion
2. Text extraction
3. Chunking
4. Embedding generation
5. Vector storage in Pinecone

Phase 2: Question Answering (Online)

1. User query
2. Query embedding
3. Similarity search (k-NN)
4. Context retrieval
5. LLM response generation
6. Conversational memory update

---

4. Data Ingestion & Preprocessing

4.1 PDF Ingestion

- Medical books and documents are loaded in PDF format
- PDFs serve as the **authoritative knowledge source**

4.2 Text Extraction

- `pypdf` is used to extract raw text from PDFs
- Metadata (page number, document source) is preserved for traceability

4.3 Document Chunking

- Extracted text is split into smaller overlapping chunks
- Chunking improves:
  - Retrieval accuracy
  - Embedding quality
  - Context relevance
- Typical chunk size: **500–1000 tokens**

---

5. Embedding Generation

- **Sentence-Transformers** is used to convert text chunks into dense vectors
- Each chunk is mapped to a high-dimensional embedding
- Embeddings capture **semantic meaning**, not just keywords

Why embeddings?

- Enable semantic similarity search
- Handle synonyms and paraphrased queries
- Improve recall in medical QA

---

6. Vector Database (Pinecone)

6.1 Pinecone Setup

- Pinecone is used as a **managed vector database**
- Stores:
    - Embeddings
    - Metadata (source, chunk ID)

6.2 Similarity Search

- Uses **k-Nearest Neighbors (k-NN)**
- Retrieves top-K most relevant chunks for a query
- Search type: **cosine similarity**

Benefits

- Low-latency retrieval
- Scales to large medical corpora
- Cloud-native and production-ready

---

7. Retrieval-Augmented Generation (RAG)

RAG Workflow

1. User submits a question
2. Question is embedded
3. Pinecone retrieves relevant chunks
4. Retrieved context is injected into the prompt
5. Gemini generates a grounded response

This approach ensures:

- Reduced hallucinations
- Fact-based medical answers
- Better trustworthiness

---

8. LLM Integration (Gemini)

- **Gemini (gemini-2.5-pro)** is used as the LLM
- Integrated via `langchain-google-genai`
- Responsible for:
    - Understanding user intent
    - Synthesizing retrieved medical context
    - Generating fluent, human-like answers

Prompt Engineering

- System prompt defines medical boundaries
- Context + user query are injected dynamically
- Prevents unsafe or speculative medical responses

---

9. Conversational Memory

- Implemented using **LangChain ConversationBufferMemory**
- Stores previous interactions in the session
- Enables:
    - Follow-up questions
    - Context continuity
    - Natural dialogue flow

Example:

> User: What is acne?
> User: What causes it?

The second question is understood **in context**, not in isolation.

---

10. Backend & API Layer

Flask Backend

- Flask is used to serve the chatbot
- Routes:
    - `/` → UI
    - `/get` → chatbot API

Responsibilities

- Handle user input

- Call RAG pipeline
- Return model responses
- Manage memory and retrieval logic

---

11. Frontend (UI)

- Built using **HTML, CSS, JavaScript**
- Features:
    - Real-time chat interface
    - Clean, responsive layout
    - User-friendly message flow

---

12. Security & Configuration

- Sensitive credentials managed via **environment variables**
- `.env` file for local development
- Secrets stored in **GitHub Actions Secrets**:
    - Pinecone API key
    - Google API key
    - AWS credentials

No hard-coded secrets in source code.

---

13. Containerization (Docker)

- Application is containerized using Docker
- Ensures:
    - Consistent runtime environment
    - Easy deployment
    - Platform independence

Dockerfile responsibilities

- Install dependencies
- Copy application code
- Run Flask app on port 8080

---

14. CI/CD Pipeline (GitHub Actions)

Continuous Integration

- Triggered on push to `main`
- Steps:
    1. Build Docker image
    2. Tag image
    3. Push image to AWS ECR

Continuous Deployment

- EC2 configured as a **self-hosted GitHub runner**
- Automatically:
    - Pulls latest image from ECR
    - Runs container
    - Exposes application on port 8080

Benefits

- Zero manual deployment
- Faster iteration
- Production-grade workflow

---

15. Cloud Deployment (AWS)

AWS Services Used

- **EC2** – Application hosting
- **ECR** – Docker image registry
- **IAM** – Secure access control

Advantages

- Scalable
- Secure
- Industry-standard cloud deployment

---

16. Tech Stack

- **Programming Language:** Python
- **Frameworks:** LangChain, Flask
- **LLM:** Gemini (Google Generative AI)
- **Embeddings:** Sentence-Transformers
- **Vector DB:** Pinecone

- **Containerization:** Docker
- **CI/CD:** GitHub Actions
- **Cloud:** AWS EC2 & ECR

---

17. Results & Outcomes

- Accurate, context-aware medical responses
- Reduced hallucinations via RAG
- Smooth multi-turn conversations
- Fully automated deployment pipeline
- Production-ready architecture

---

18. Future Enhancements

- Role-based access (doctor vs patient)
- Source citation highlighting
- Multilingual support
- Feedback-based retrieval ranking
- HIPAA-compliant storage
- Analytics dashboard for usage tracking

---

19. Conclusion

This project demonstrates a **full-stack, production-grade AI system** combining **LLMs, vector databases, cloud infrastructure, and CI/CD**. It showcases strong skills in **Generative AI, system design, MLOps, and cloud deployment**, making it highly relevant for **AI Engineer, GenAI Developer, and ML Engineer roles**.